

1. Create a static website using HTML5 with semantic elements, forms, tables, audio, video.

```
<!DOCTYPE html>
<html>
<head>
<title>Static Website</title>

<style>

body{
    font-family:Arial;
    background:#f0f8ff;
    margin:20px;
}

header{
    background:#0077b6;
    color:white;
    padding:15px;
    text-align:center;
}

section{
    background:white;
    padding:20px;
    margin-top:20px;
    border-radius:10px;
}

table{
    width:40%;
    border-collapse:collapse;
}

th,td{
    border:1px solid gray;
    padding:10px;
    text-align:center;
    width: 33%;
}
```

```
button{
  background:#0077b6;
  color:white;
  border:none;
  padding:10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Static Website</h1>
```

```
</header>
```

```
<section>
```

```
<h2>Student Form</h2>
```

```
<form>
```

Name:

```
<input type="text"><br><br>
```

Email:

```
<input type="email"><br><br>
```

```
<button>Submit</button>
```

```
</form>
```

```
</section>
```

```
<section>
```

```
<h2>Student Table</h2>
```

```
<table>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
<tr>
<td>Ali</td>
<td>20</td>
</tr>
</table>
</section>
```

```
<section>
```

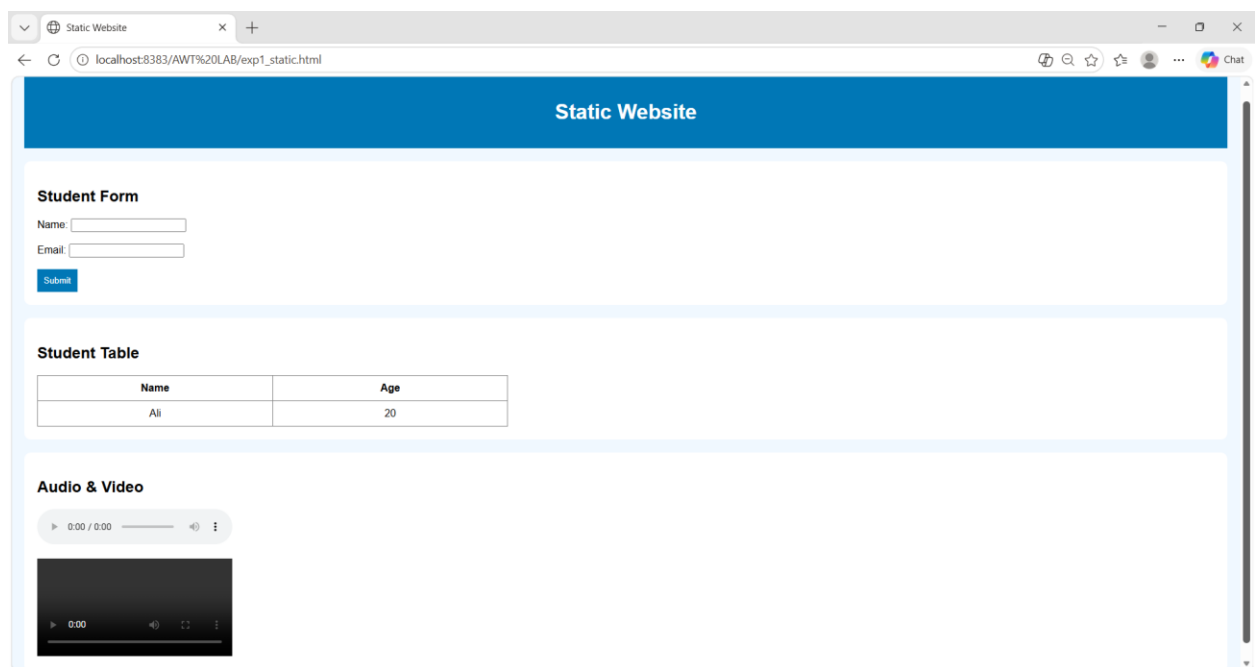
```
<h2>Audio & Video</h2>
```

```
<audio controls>
<source src="audio.mp3">
</audio>
```

```
<br><br>
```

```
<video width="300" controls>
<source src="video.mp4">
</video>
</section>
```

```
</body>
</html>
```



2. Design responsive web pages using CSS3, Flexbox, Grid, and media queries.

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Web Page</title>
<style>

body{
font-family:Arial;
background:#f5f5f5;
padding:20px;
}

h2{
text-align:center;
color:#6a1b9a;
}

.container{
display:grid;
grid-template-columns:1fr 1fr 1fr;
gap:20px;
margin-top:20px;
}

.card{
background:white;
padding:20px;
border-radius:10px;
text-align:center;
box-shadow:0 0 10px lightgray;
}
```

```
.card img{  
width:100%;  
height:150px;  
border-radius:10px;  
}
```

```
.card h3{  
color:#6a1b9a;  
}
```

```
.flex{  
display:flex;  
gap:20px;  
margin-top:20px;  
}
```

```
.item{  
flex:1;  
background:#ffcc80;  
padding:20px;  
text-align:center;  
border-radius:10px;  
font-size:20px;  
}
```

```
@media(max-width:700px){  
.container{  
grid-template-columns:1fr;  
}  
.flex{  
flex-direction:column;  
}  
}
```

</style>

</head>

<body>

<h2>Responsive Web Design</h2>

<div class="container">

<div class="card">

<h3>HTML5</h3>

<p>Semantic structure for web pages.</p>

</div>

<div class="card">

<h3>CSS3</h3>

<p>Responsive and attractive styling.</p>

</div>

<div class="card">

<h3>JavaScript</h3>

<p>Interactive web page functionality.</p>

</div>

</div>

<div class="flex">

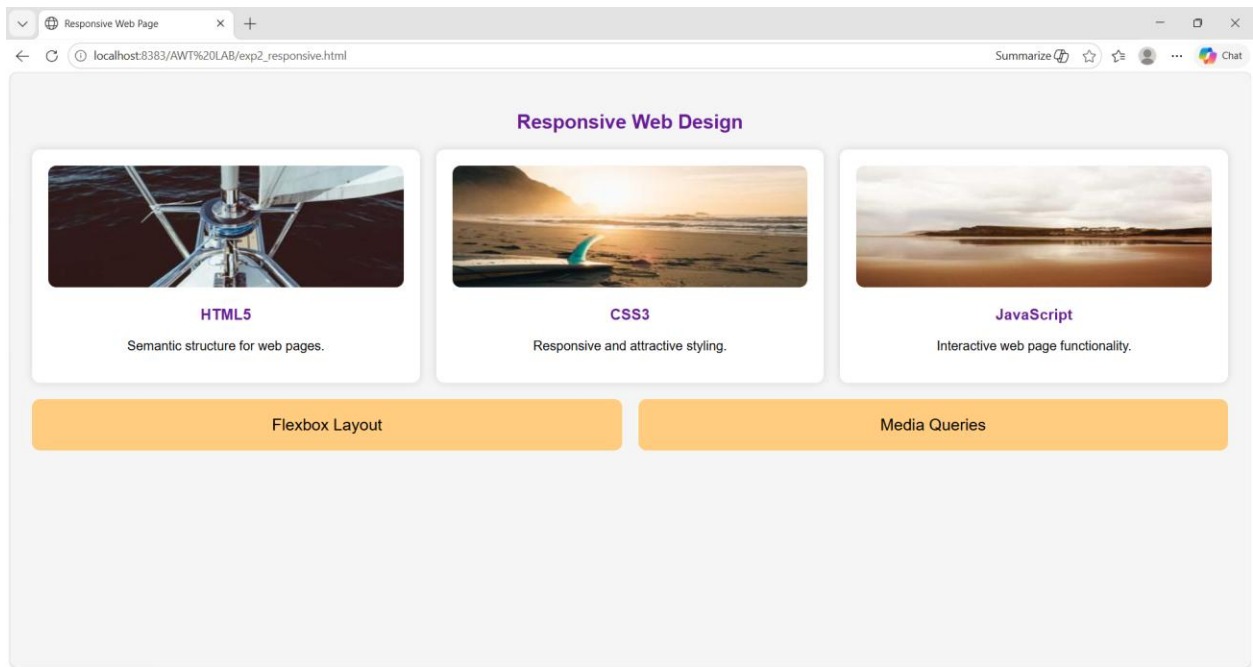
<div class="item">Flexbox Layout</div>

<div class="item">Media Queries</div>

</div>

</body>

</html>



3. Implement client-side validation for forms using JavaScript.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body{  
    margin-top: 250px;  
    font-family:Arial;  
    background:#eef;  
}
```

```
form{  
    width:350px;  
    margin:auto;  
    background:white;  
    padding: 20px;  
    padding-left:20px;
```

```
padding-right:40px;
border-radius:10px;
}
```

```
input{
width:100%;
padding:8px;
margin-top:5px;
}
```

```
button{
background:#0077b6;
color:white;
padding:10px;
border:none;
margin-top:10px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2 align="center">
```

```
Student Registration
```

```
</h2>
```

```
<form onsubmit="return validate()">
```

```
Name:
```

```
<input type="text" id="name"><br><br>
```

```
Email:
```

```
<input type="text" id="email"><br><br>
```

```
Phone:
```

```
<input type="text" id="phone"><br><br>
```

```
<button>Submit</button>
```



```
</form>
```

```
<script>
```

```
function validate(){
```

```
let name=document.getElementById("name").value;
```

```
let email=document.getElementById("email").value;
```

```
let phone=document.getElementById("phone").value;
```

```
if(name==""){
```

```
    alert("Enter Name");
```

```
    return false;
```

```
}
```

```
if(email==""){
```

```
    alert("Enter Email");
```

```
    return false;
```

```
}
```

```
if(phone.length!=10){
```

```
    alert("Phone number must be 10 digits");
```

```
    return false;
```

```
}
```

```
alert("Form Submitted Successfully");
```

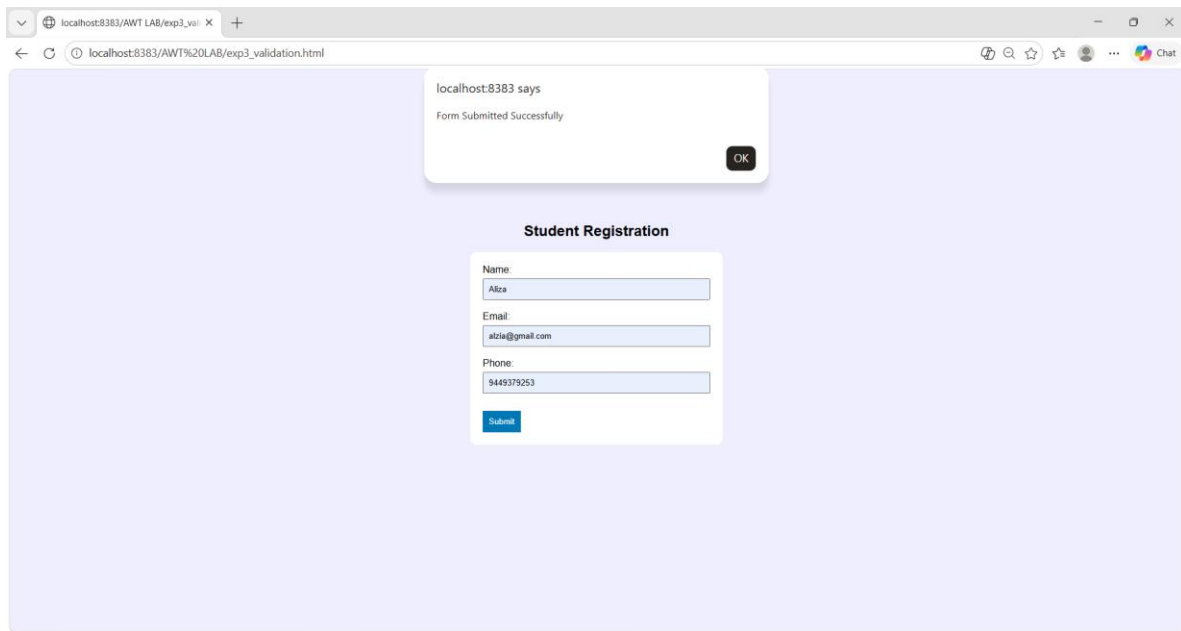
```
return true;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



4. Develop interactive web pages using DOM manipulation and event handling.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body{  
    font-family:Arial;  
    text-align:center;  
    background:#eef;  
    margin-top:80px;  
}
```

```
.box{  
    background:white;  
    width:400px;  
    margin:auto;  
    padding:30px;
```

```
border-radius:10px;
}
```

```
button{
  background:#0077b6;
  color:white;
  border:none;
  padding:10px;
  margin:5px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box">
```

```
<h1 id="text">Welcome Student</h1>
```

```

```

```
<br><br>
```

```
<button onclick="changeText()">
```

```
Change Text
```

```
</button>
```

```
<button onclick="changeColor()">
```

```
Change Color
```

```
</button>
```

```
<button onclick="hideImage()">
```

```
Hide Image
```

```
</button>
```

```
</div>
```

```
<script>

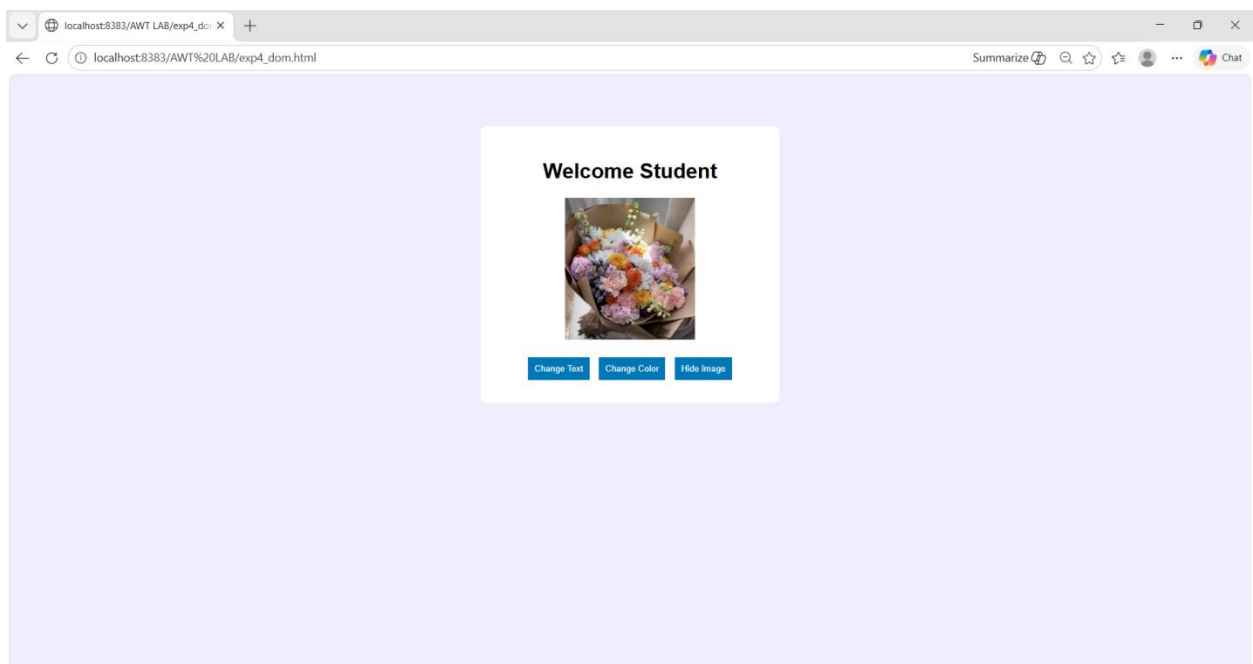
function changeText(){
document.getElementById("text").innerHTML=
"Advanced Web Technology";
}

function changeColor(){
document.body.style.background="lightyellow";
}

function hideImage(){
document.getElementById("image").style.display="none";
}

</script>

</body>
</html>
```



5. Implement ES6 features such as arrow functions, promises, and modules.

```
<!DOCTYPE html>
<html>
<head>
<title>ES6 Features</title>
<style>
```

```
body{
font-family:Arial;
background:#fff3e0;
padding:30px;
}
```

```
.box{
width:650px;
margin:auto;
background:white;
padding:20px;
border-radius:10px;
}
```

```
h2{
text-align:center;
color:#e65100;
}
```

```
div{
background:#fbe9e7;
padding:15px;
margin-top:15px;
border-radius:10px;
}
```

```
button{
width:100%;
padding:10px;
background:#e65100;
color:white;
border:none;
font-size:18px;
margin-top:10px;
}
```

```
p{
background:white;
padding:10px;
font-size:18px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box">
```

```
<h2>ES6 Features</h2>
```

```
<div>
```

```
<h3>Arrow Function</h3>
```

```
<button onclick="arrow()">Run</button>
```

```
<p id="a"></p>
```

```
</div>
```

```
<div>
```

```
<h3>Promise</h3>
```

```
<button onclick="promise()">Run</button>
```

```
<p id="b"></p>
```

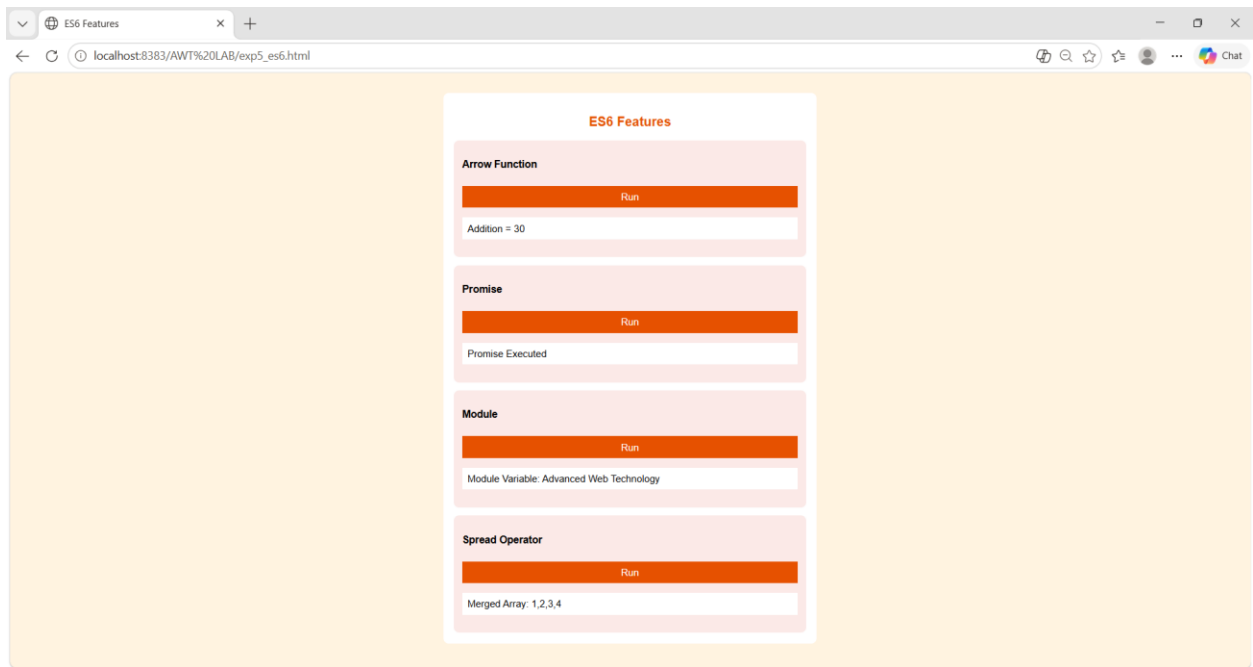
```
</div>
```

```
<div>
<h3>Module</h3>
<button onclick="module1()">Run</button>
<p id="c"></p>
</div>
```

```
<div>
<h3>Spread Operator</h3>
<button onclick="spread()">Run</button>
<p id="d"></p>
</div>
```

```
</div>
```

```
<script type="module">
export let subject="Advanced Web Technology";
const add=(x,y)=>x+y;
window.arrow=()=>a.innerHTML=
"Addition = "+add(10,20);
window.promise=()=>{
b.innerHTML="Loading...";
new Promise(r=>setTimeout(()=>r("Promise Executed"),1000))
.then(data=>b.innerHTML=data);
}
window.module1=()=>c.innerHTML=
"Module Variable: "+subject;
window.spread=()=>{
let x=[1,2],y=[3,4];
d.innerHTML="Merged Array: "+[...x,...y];
}
</script>
</body>
</html>
```



6. Use Fetch API/AJAX to retrieve and display dynamic data.

```
<!DOCTYPE html>
<html>
<head>
<title>Fetch API</title>
<style>
body{
  font-family:Arial;
  background:#eef;
  padding:20px;
}

.container{
  width:400px;
  margin:auto;
  background:white;
  padding:20px;
}
```



```
input{
  width:100%;
  padding:8px;
  box-sizing:border-box;
}
```

```
button{
  background:#0077b6;
  color:white;
  border:none;
  padding:10px;
  margin-top:10px;
}
```

```
.card{
  background:#f4f4f4;
  padding:15px;
  margin-top:20px;
}
```

```
img{
  width:100%;
  height:180px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<h2>Country Information</h2>
```

```
Enter Country Name:
```

```
<input type="text" id="country">
<button onclick="loadCountry()">
```

Get Details

```
</button>
```

```
<div id="result"></div>
</div>
```

```
<script>
function loadCountry(){
let countryName=
document.getElementById("country").value;
fetch("https://restcountries.com/v3.1/name/"+countryName)
```

```
.then(response=>response.json())
.then(data=>{
let country=data[0];
document.getElementById("result").innerHTML=
```

```
"<div class='card'>" +
```

```
"<img src='"+country.flags.png+"'>" +
"<h3>"+country.name.common+"</h3>" +
"<p>Capital: "+country.capital+"</p>" +
"<p>Region: "+country.region+"</p>" +
"<p>Population: "+country.population+"</p>" +
"</div>";
```

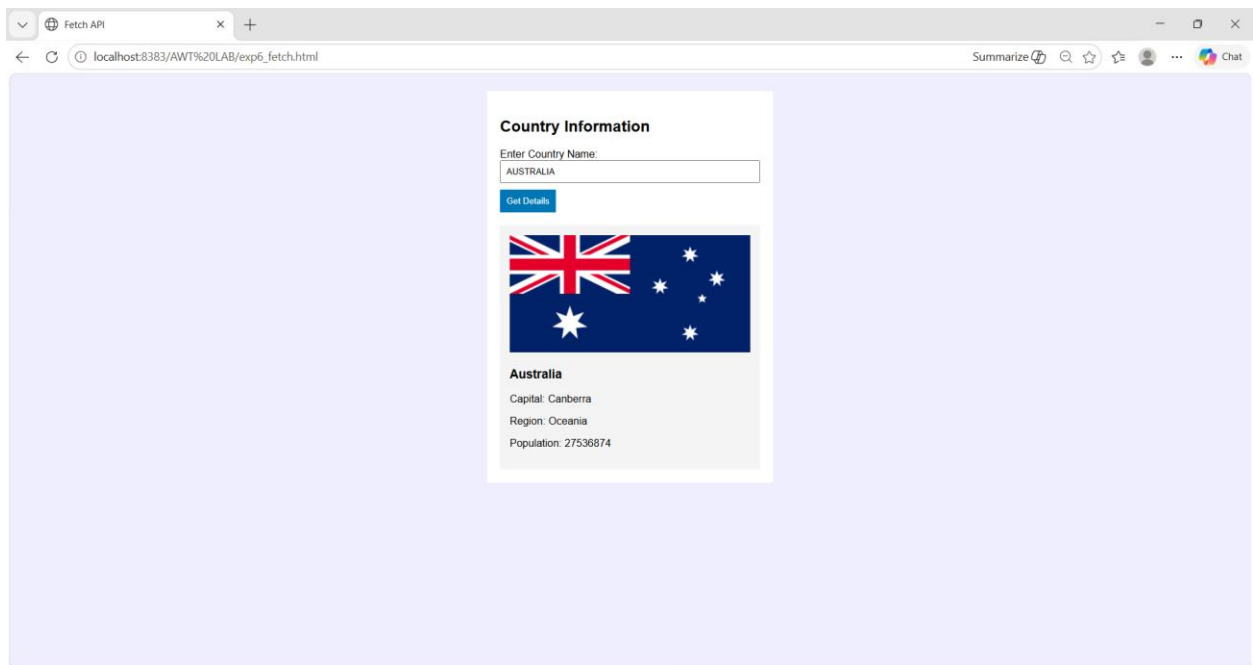
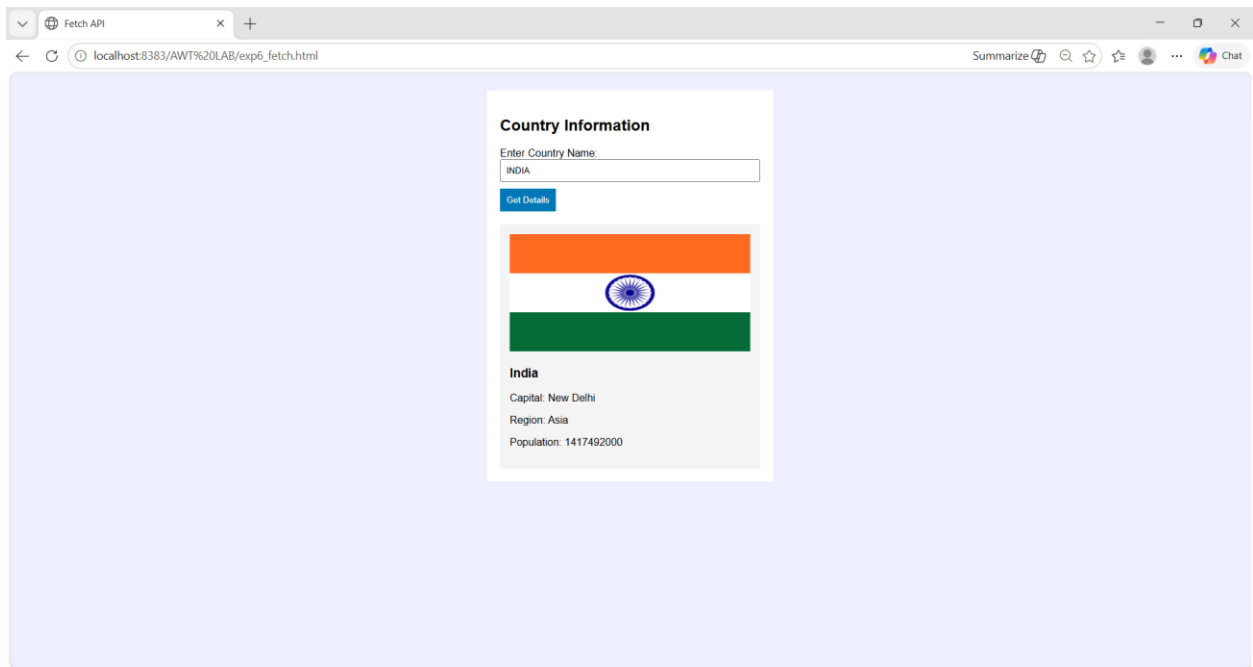
```
});
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



7. Implement client-side storage using Cookies, LocalStorage, and SessionStorage.

```
<!DOCTYPE html>
<html>
<head>
<title>Client Side Storage</title>
<style>
```

```
body{
font-family:Arial;
background:#e3f2fd;
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
```

```
.box{
width:600px;
background:white;
padding:40px;
border-radius:15px;
border:4px solid #42a5f5;
}
```

```
h2{
text-align:center;
color:#1565c0;
font-size:42px;
}
```

```
label{
font-size:28px;
```

```
font-weight:bold;
}
```

```
input{
width:100%;
padding:15px;
margin-top:15px;
box-sizing:border-box;
font-size:24px;
border:2px solid #90caf9;
}
```

```
button{
width:100%;
padding:15px;
margin-top:15px;
border:none;
color:white;
font-size:22px;
border-radius:8px;
}
```

```
.local{background:#43a047;}
.session{background:#fb8c00;}
.cookie{background:#8e24aa;}
.show{background:#1565c0;}
```

```
p{
background:#f1f8e9;
padding:20px;
margin-top:25px;
font-size:24px;
border-radius:8px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box">
```

```
<h2>Client Side Storage</h2>
```

```
<label>Enter Name:</label>
```

```
<input type="text" id="name">
```

```
<button class="local" onclick="saveLocal()">
```

```
Save to LocalStorage
```

```
</button>
```

```
<button class="session" onclick="saveSession()">
```

```
Save to SessionStorage
```

```
</button>
```

```
<button class="cookie" onclick="saveCookie()">
```

```
Save Cookie
```

```
</button>
```

```
<button class="show" onclick="showData()">
```

```
Show Data
```

```
</button>
```

```
<p id="output">
```

```
Stored data will appear here
```

```
</p>
```

```
</div>
```

```
<script>
```

```
function saveLocal(){
```

```
let name=document.getElementById("name").value;
localStorage.setItem("username",name);
alert("Saved in LocalStorage");
}
```

```
function saveSession(){
let name=document.getElementById("name").value;
sessionStorage.setItem("username",name);
alert("Saved in SessionStorage");
}
```

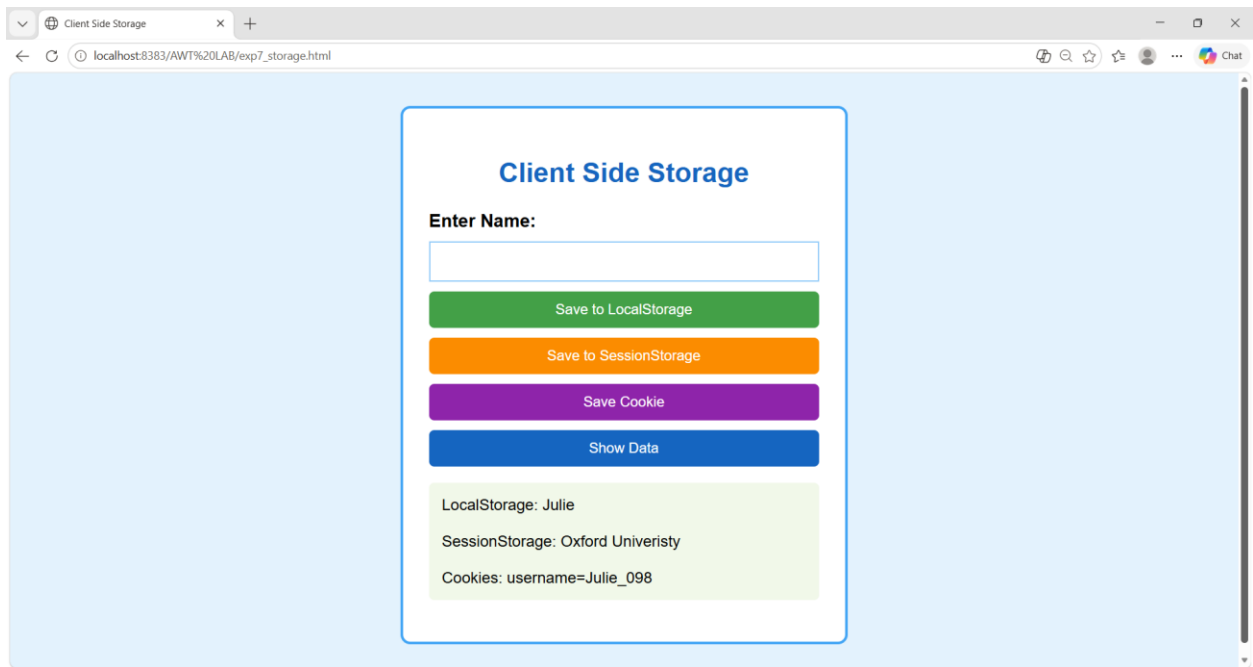
```
function saveCookie(){
let name=document.getElementById("name").value;
document.cookie="username="+name;
alert("Cookie Saved");
}
```

```
function showData(){
let local=localStorage.getItem("username");
let session=sessionStorage.getItem("username");
document.getElementById("output").innerHTML=
"LocalStorage: "+local+
"<br><br>SessionStorage: "+session+
"<br><br>Cookies: "+document.cookie;
}
```

```
</script>
```

```
</body>
```

```
</html>
```



8. Develop server-side scripts to handle GET and POST requests.

Software Requirements

- NetBeans IDE
- Node.js
- Express.js
- Web Browser

PROGRAM

server.js

```
const express=require("express");
const app=express();
app.use(express.urlencoded({extended:true}));

// GET Request
app.get("/",(req,res)=>{
res.send("<h1 style='color:purple;font-size:40px;text-align:center'>GET Request Successful</h1>");
});

// POST Request
app.post("/submit",(req,res)=>{
```



```
res.send("<h1 style='color:green;font-size:40px;text-align:center'>POST Request Received:  
"+req.body.name+"</h1>");  
});
```

```
// Server  
app.listen(3000,()=>{  
  console.log("Server Running on Port 3000");  
});
```

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
<title>GET and POST</title>
```

```
<style>  
body{  
  font-family:Arial;  
  background:#f5f5f5;  
  text-align:center;  
  padding:50px;  
}
```

```
.box{  
  background:white;  
  width:400px;  
  margin:auto;  
  padding:30px;  
  border-radius:10px;  
}
```

```
input,button{  
  width:90%;
```

```
padding:10px;
margin:10px;
font-size:18px;
}
```

```
button{
background:#8e44ad;
color:white;
border:none;
}
```

```
</style>
</head>
```

```
<body>
<div class="box">
<h2>GET and POST Request</h2>
<form action="http://localhost:3000" method="GET">
<button> GET Request </button>
</form>
```

```
<form action="http://localhost:3000/submit" method="POST">
<input type="text" name="name" placeholder="Enter Name">
<button> POST Request </button>
</form>
</div>
</body>
</html>
```

PROCEDURE / EXECUTION STEPS

Step 1:

Open NetBeans IDE and create a project named:
AWT LAB

Inside the Site Root folder, create two files:

server.js

index.html

Write the Node.js Express code in server.js and the HTML code in index.html.

Step 2:

Open the project folder location in File Explorer and open the: public_html folder.

Click on the address bar, type:

cmd

and press Enter to open Command Prompt in the same folder.

Step 3:

Initialize the Node.js project and install Express.js using:

npm init -y

npm install express

Step 4:

Run the server using: node server.js

The terminal displays: Server Running on Port 3000

Step 5:

Go back to NetBeans, right-click on: index.html and select: Run File

Browser opens automatically.

Click on the: GET Request button.

Output displayed:

GET Request Successful

Enter a name in the textbox.

Example: Department of CSE

Click on the: POST Request button.

Output displayed:

POST Request Received: Department of CSE

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26280.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>npm init -y
Wrote to C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html\package.json:

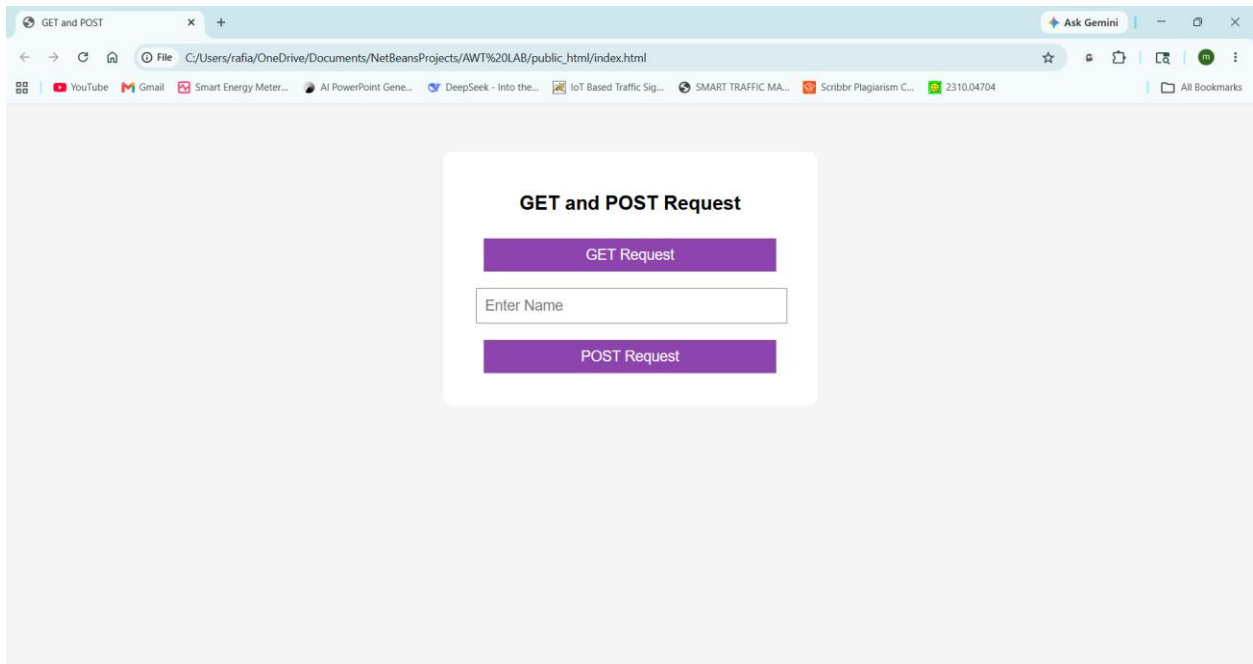
{
  "name": "public_html",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {},
  "description": ""
}

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>npm install express
up to date, audited 67 packages in 2s

24 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>node server.js
Server Running on Port 3000
```





GET Request Successful



GET and POST Request

GET Request

POST Request



POST Request Received: Department of CSE

9. Design and connect a database (MySQL/MongoDB) to perform CRUD operations.

Software Requirements

MySQL Server

MySQL Workbench

Installation of MySQL

- Download and install MySQL Server and MySQL Workbench from the MySQL official website.
- Configure the root user account and password during installation.
- Launch MySQL Workbench and connect to the local MySQL server to execute SQL queries.

Procedure

Step 1: Create Database

Query:

```
CREATE DATABASE CollegeDB;
```

Output: Database created successfully.

Step 2: Select Database

Query:

```
USE CollegeDB;
```

Output: Database changed.

Step 3: Create Student Table

Query:

```
CREATE TABLE Student (
```

```
USN VARCHAR(15) PRIMARY KEY,  
Student_Name VARCHAR(50),  
Department VARCHAR(30),  
Semester INT,  
CGPA DECIMAL(3,2),  
Email VARCHAR(50)  
);
```

Output: Table created successfully.

CREATE Operation

Example 1: Insert First Student Record

Query:

```
INSERT INTO Student  
VALUES ('25KB02MTS01',  
        'Aliza Mahvash', 'Computer Science', 2, 8.75, 'aliza@gmail.com');
```

Output: 1 row inserted.

Example 2: Insert Second Student Record

Query:

```
INSERT INTO Student  
VALUES ('25KB02MTS02', 'Ayesha Khan', 'Computer Science', 2, 9.10, 'ayesha@gmail.com');
```

Output: 1 row inserted.

READ Operation

Example 1: Display All Student Records

Query:

```
SELECT * FROM Student;
```

Output:

USN	Student_Name	Department	Semester	CGPA	Email
25KB02MTS01	Aliza Mahvash	Computer Science	2	8.75	aliza@gmail.com
25KB02MTS02	Ayesha Khan	Computer Science	2	9.10	ayesha@gmail.com

Example 2: Display Students of Computer Science Department

Query:

SELECT * FROM Student

WHERE Department='Computer Science';

Output: Both student records are displayed.

UPDATE Operation

Example 1: Update Student CGPA

Query:

UPDATE Student

SET CGPA = 9.20

WHERE USN = '25KB02MTS01';

Output: 1 row updated.

Example 2: Verify Updated Record

Query:

SELECT * FROM Student

WHERE USN='25KB02MTS01';

Output:

USN	Student_Name	Department	Semester	CGPA	Email
25KB02MTS01	Aliza Mahvash	Computer Science	2	9.20	aliza@gmail.com

DELETE Operation

Example 1: Delete Student with USN 25KB02MTS02

Query:

DELETE FROM Student

WHERE USN = '25KB02MTS02';

Output: 1 row deleted.

Example 2: Delete Student with USN 25KB02MTS01

Query:

DELETE FROM Student

WHERE USN = '25KB02MTS01';

Output: 1 row deleted.

Verification

Query:

SELECT * FROM Student;

Output: Empty set (0 rows).

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Query 1 x collegedb

```

17 'aliza@gmail.com');
18 INSERT INTO Student
19 VALUES ('25KB02MTS02',
20 'Ayesha Khan',
21 'Computer Science',
22 2,
23 9.10,
24 'ayesha@gmail.com');
25 SELECT * FROM Student;

```

Result Grid

USN	Student_Name	Department	Semester	CGPA	Email
25KB02MTS01	Aliza Mahvash	Computer Science	2	8.75	aliza@gmail.com
25KB02MTS02	Ayesha Khan	Computer Science	2	9.10	ayesha@gmail.com

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

Student 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	23:32:31	drop database collegedb	0 row(s) affected	0.062 sec
2	23:32:39	CREATE DATABASE CollegeDB	1 row(s) affected	0.016 sec
3	23:32:56	USE CollegeDB	0 row(s) affected	0.015 sec
4	23:33:23	CREATE TABLE Student (USN VARCHAR(15) PRIMARY KEY, Student_Name VARCHAR(50), Depart...	0 row(s) affected	0.047 sec
5	23:36:18	INSERT INTO Student VALUES ('25KB02MTS01', 'Aliza Mahvash', 'Computer Science', 2, ...	1 row(s) affected	0.015 sec
6	23:37:23	INSERT INTO Student VALUES ('25KB02MTS02', 'Ayesha Khan', 'Computer Science', 2, 9...	1 row(s) affected	0.016 sec
7	23:37:56	SELECT * FROM Student LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Query 1 x collegedb

```

21 'Computer Science',
22 2,
23 9.10,
24 'ayesha@gmail.com');
25 SELECT * FROM Student;
26 UPDATE Student
27 SET CGPA = 9.20
28 WHERE USN = '25KB02MTS01';
29

```

Result Grid

USN	Student_Name	Department	Semester	CGPA	Email
25KB02MTS01	Aliza Mahvash	Computer Science	2	9.20	aliza@gmail.com
25KB02MTS02	Ayesha Khan	Computer Science	2	9.10	ayesha@gmail.com

Administration Schemas

Information

No object selected

Object Info Session

Query Completed

Student 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
2	23:32:39	CREATE DATABASE CollegeDB	1 row(s) affected	0.016 sec
3	23:32:56	USE CollegeDB	0 row(s) affected	0.015 sec
4	23:33:23	CREATE TABLE Student (USN VARCHAR(15) PRIMARY KEY, Student_Name VARCHAR(50), Depart...	0 row(s) affected	0.047 sec
5	23:36:18	INSERT INTO Student VALUES ('25KB02MTS01', 'Aliza Mahvash', 'Computer Science', 2, ...	1 row(s) affected	0.015 sec
6	23:37:23	INSERT INTO Student VALUES ('25KB02MTS02', 'Ayesha Khan', 'Computer Science', 2, ...	1 row(s) affected	0.016 sec
7	23:37:56	SELECT * FROM Student LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
8	23:39:31	UPDATE Student SET CGPA = 9.20 WHERE USN = '25KB02MTS01'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
9	23:39:51	SELECT * FROM Student LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

10. Develop RESTful APIs for a Simple Application.

Software Requirements

- NetBeans IDE
- Node.js
- Express.js
- Web Browser

PROGRAM

api.js

```
const express=require("express");
const app=express();
app.use((req,res,next)=>{
res.header("Access-Control-Allow-Origin","*");
res.header("Access-Control-Allow-Headers","*");
res.header("Access-Control-Allow-Methods","GET,POST,DELETE");
next();
});

app.use(express.json());

let students=[
{id:1,name:"Aliza"}
];

// GET
app.get("/students",(req,res)=>{
res.json(students);
});

// POST
app.post("/students",(req,res)=>{
let newStudent={
```

```
id:students.length+1,  
name:req.body.name  
};
```

```
students.push(newStudent);  
res.json(students);  
});
```

```
// DELETE  
app.delete("/students/:id",(req,res)=>{  
  let id=parseInt(req.params.id);  
  students=students.filter(s=>s.id!==id);  
  res.json(students);  
});
```

```
app.listen(3000,()=>{  
  console.log("REST API Running on Port 3000");  
});
```

index.html

```
<!DOCTYPE html>  
<html>  
<head>  
<title>REST API Demo</title>
```

```
<style>
```

```
body{  
  font-family:Arial;  
  background:#fff8e1;  
  text-align:center;  
  padding:50px;  
}
```

```
.box{
background:white;
width:500px;
margin:auto;
padding:25px;
border-radius:10px;
border:3px solid #ff9800;
}
```

```
h2{
color:#e65100;
}
```

```
input,button{
width:90%;
padding:10px;
margin:10px;
font-size:18px;
}
```

```
button{
background:#ff9800;
color:white;
border:none;
cursor:pointer;
}
```

```
p{
font-size:18px;
font-weight:bold;
word-wrap:break-word;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box">
```

```
<h2>REST API Testing</h2>
```

```
<input type="text" id="name" placeholder="Enter Name">
```

```
<input type="number" id="id" placeholder="Enter ID to Delete">
```

```
<button onclick="getData()">GET</button>
```

```
<button onclick="postData()">POST</button>
```

```
<button onclick="deleteData()">DELETE</button>
```

```
<p id="output"></p>
```

```
</div>
```

```
<script>
```

```
function getData(){  
  fetch("http://localhost:3000/students")  
  .then(res=>res.json())  
  .then(data=>{  
    document.getElementById("output").innerHTML=  
    JSON.stringify(data);  
  });  
}
```

```
function postData(){  
  let name=document.getElementById("name").value;  
  fetch("http://localhost:3000/students",{  
    method:"POST",  
    headers:{  
      "Content-Type":"application/json"  
    },  
    body:JSON.stringify({name:name})  
  })
```

```
.then(res=>res.json())  
.then(data=>{
```

```
document.getElementById("output").innerHTML=
JSON.stringify(data);
});
}
```

```
function deleteData(){
let id=document.getElementById("id").value;
fetch("http://localhost:3000/students/"+id,{
method:"DELETE"
})
.then(res=>res.json())
.then(data=>{
document.getElementById("output").innerHTML=
JSON.stringify(data);
});
}
```

</script>

</body>

</html>

PROCEDURE / EXECUTION STEPS

Step 1:

Open NetBeans IDE and create a project named:

AWT LAB

Inside the Site Root folder, create two files:

api.js

index.html

Write the Node.js REST API code in api.js and the HTML code in index.html.

Step 2:

Open the project folder location in File Explorer and open the public_html folder.

Click on the address bar, type: cmd and press Enter to open Command Prompt in the same folder.

Step 3:

Initialize the Node.js project and install Express.js using:

```
npm init -y
```

```
npm install express
```

Step 4:

Run the REST API server using: `node api.js`

The terminal displays:

REST API Running on Port 3000

Step 5:

Go back to NetBeans, right-click on:

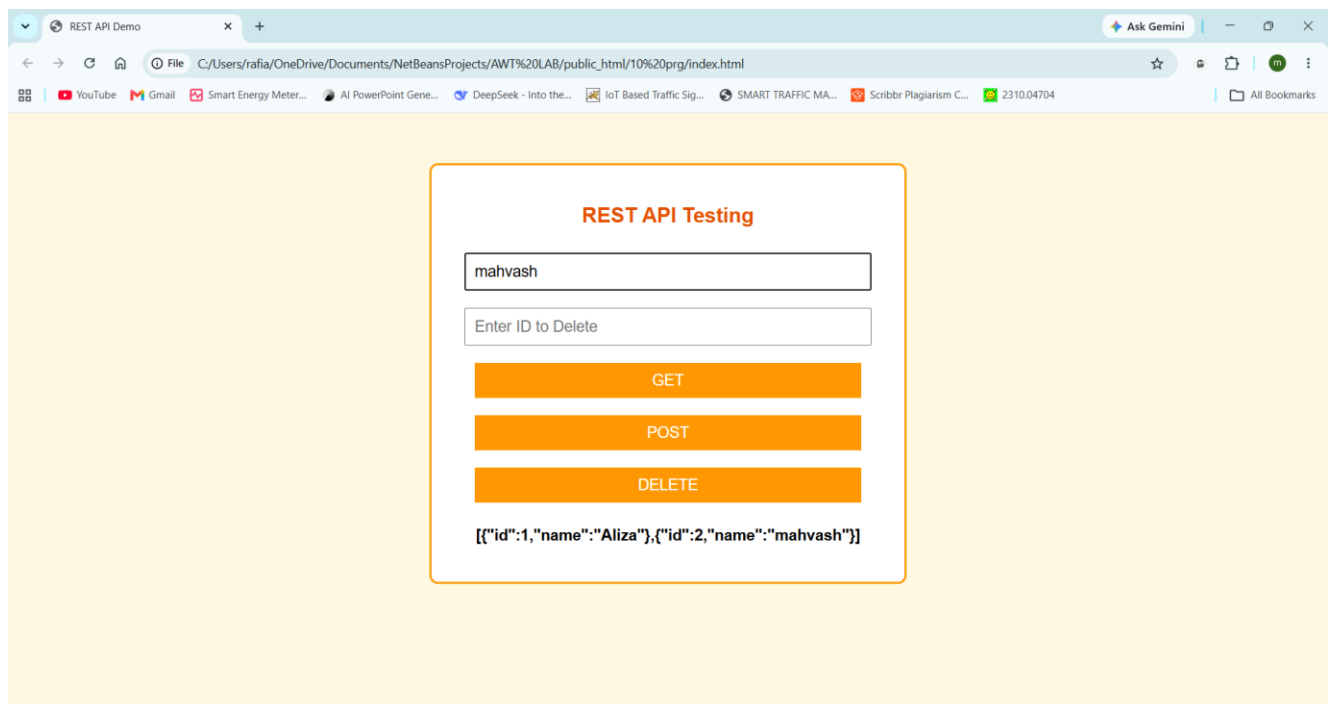
index.html

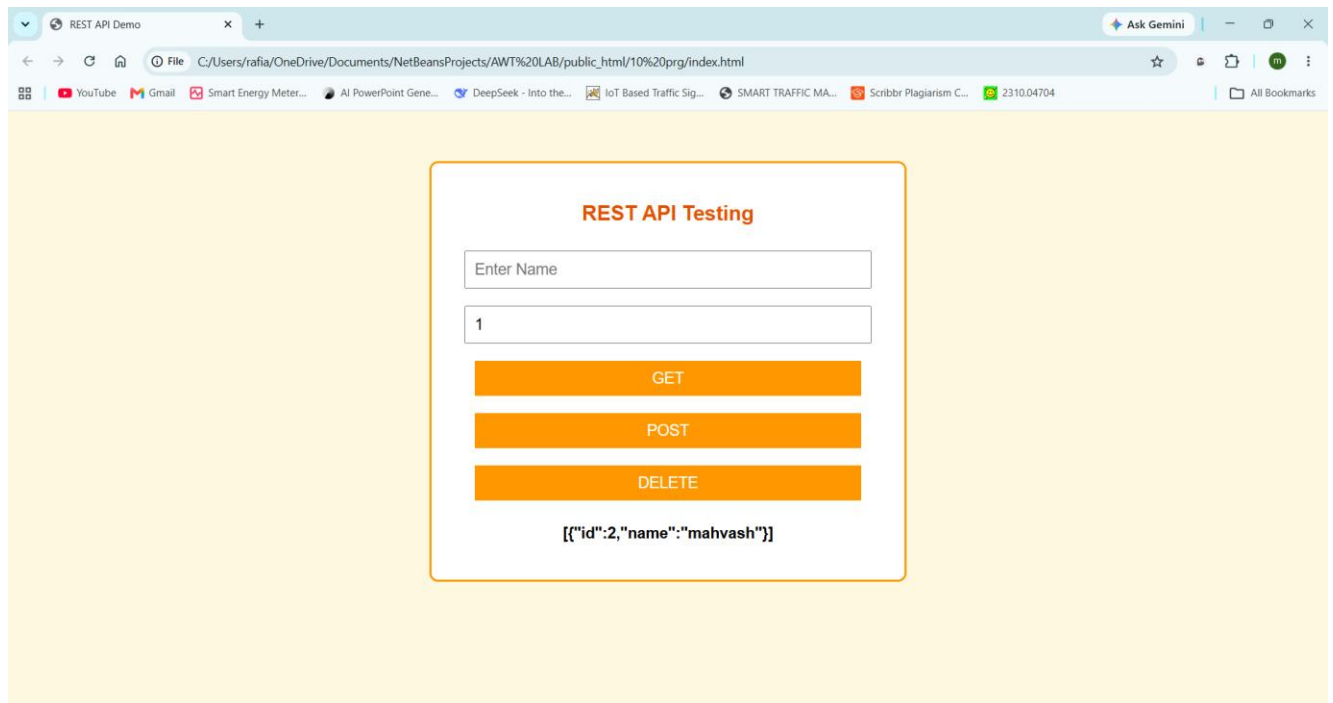
Select:

Run File

The browser opens automatically.

- Click GET button to retrieve all student records.
- Enter a name and click POST button to add a new student.
- Enter a Student ID and click DELETE button to remove a student record.





11. Version Control Using Git and GitHub. Deploy a Simple Web Application

Software Requirements

- NetBeans IDE
- Git
- GitHub Account
- Web Browser

PROCEDURE / EXECUTION STEPS

Step 1:

Install Git from the Git website and create a GitHub account.

Step 2:

Open NetBeans IDE and open the project folder:

AWT LAB

Ensure all experiment files are available inside the public_html folder.

Step 3:

Open the public_html folder in File Explorer.

Click on the address bar, type: cmd and press Enter to open Command Prompt in the project folder.

Step 4:

Initialize the Git repository using:

```
git init
```

Output: Initialized empty Git repository

Step 5:

Add all project files to Git using:

```
git add .
```

Verify the files using:

```
git status
```

All project files will appear under:

Changes to be committed

Step 6:

Create the first commit using:

```
git commit -m "First Commit"
```

Output: [master (root-commit)] First Commit

Step 7:

Login to GitHub and create a new repository named:

awt-lab

Select Public Repository and click Create Repository.

Step 8:

Connect the local repository to GitHub using:

```
git remote add origin https://github.com/Aliza-15/awt-lab.git
```

```
git branch -M main
```

```
git push -u origin main
```

Authorize Git Credential Manager when prompted.

Step 9:

After successful upload, verify that all project files are visible in the GitHub repository.

Step 10:

Deploy the project using GitHub Pages.

Open:

Repository → Settings → Pages

Under Build and Deployment:

Select:

Source → Deploy from a branch

Choose:

Branch → main

Folder → /(root)

Click: Save

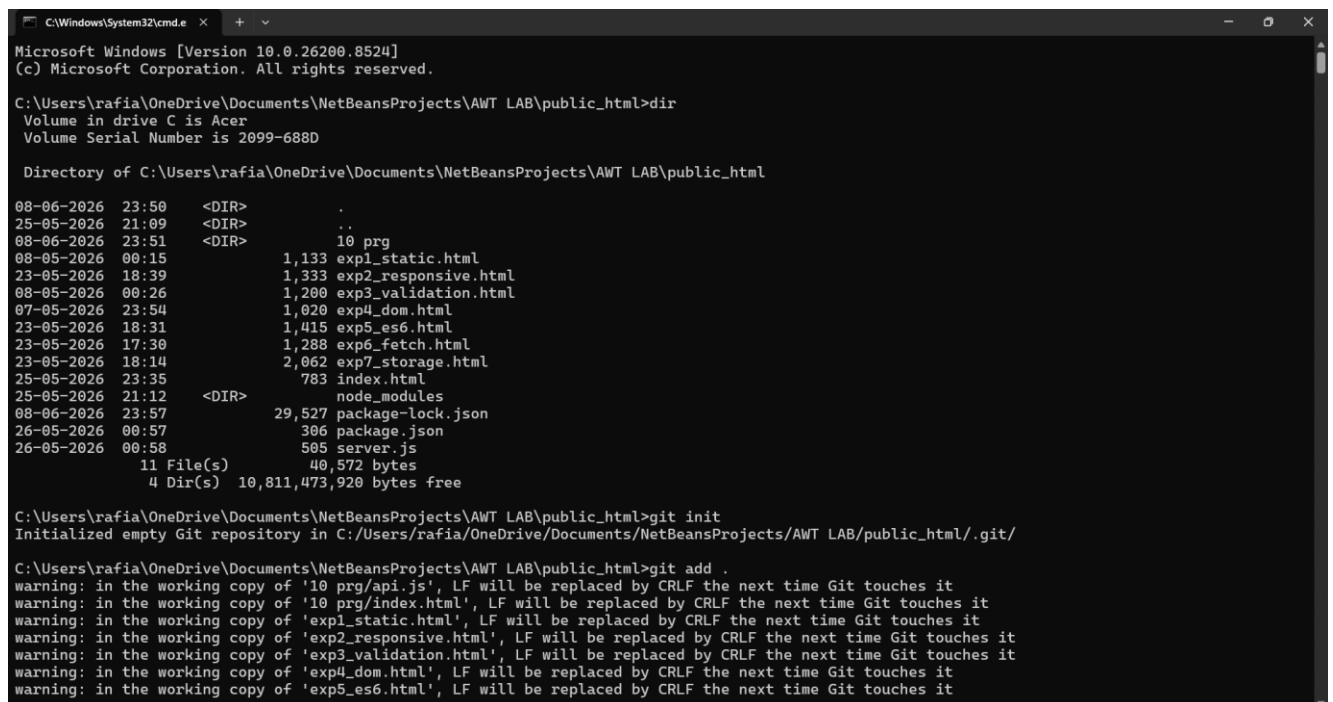
Step 11:

Wait for deployment to complete.

GitHub generates a live website URL:

<https://aliza-15.github.io/awt-lab/>

Open the URL in the browser to verify the deployment.



```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>dir
Volume in drive C is Acer
Volume Serial Number is 2099-688D

Directory of C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html

08-06-2026  23:50    <DIR>          .
25-05-2026  21:09    <DIR>          ..
08-06-2026  23:51    <DIR>          10 prg
08-05-2026  00:15                1,133 exp1_static.html
23-05-2026  18:39                1,333 exp2_responsive.html
08-05-2026  00:26                1,200 exp3_validation.html
07-05-2026  23:54                1,020 exp4_dom.html
23-05-2026  18:31                1,415 exp5_es6.html
23-05-2026  17:30                1,288 exp6_fetch.html
23-05-2026  18:14                2,062 exp7_storage.html
25-05-2026  23:35                783  index.html
25-05-2026  21:12    <DIR>          node_modules
08-06-2026  23:57                29,527 package-lock.json
26-05-2026  00:57                306  package.json
26-05-2026  00:58                505  server.js
               11 File(s)              40,572 bytes
               4 Dir(s)  10,811,473,920 bytes free

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>git init
Initialized empty Git repository in C:/Users/rafia/OneDrive/Documents/NetBeansProjects/AWT LAB/public_html/.git/

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>git add .
warning: in the working copy of '10 prg/api.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of '10 prg/index.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'exp1_static.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'exp2_responsive.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'exp3_validation.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'exp4_dom.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'exp5_es6.html', LF will be replaced by CRLF the next time Git touches it
```

```
C:\Windows\System32\cmd.e
create mode 100644 node_modules/type-is/node_modules/content-type/package.json
create mode 100644 node_modules/type-is/package.json
create mode 100644 node_modules/unpipe/HISTORY.md
create mode 100644 node_modules/unpipe/LICENSE
create mode 100644 node_modules/unpipe/README.md
create mode 100644 node_modules/unpipe/index.js
create mode 100644 node_modules/unpipe/package.json
create mode 100644 node_modules/vary/HISTORY.md
create mode 100644 node_modules/vary/LICENSE
create mode 100644 node_modules/vary/README.md
create mode 100644 node_modules/vary/index.js
create mode 100644 node_modules/vary/package.json
create mode 100644 node_modules/wrappy/LICENSE
create mode 100644 node_modules/wrappy/README.md
create mode 100644 node_modules/wrappy/package.json
create mode 100644 node_modules/wrappy/wrappy.js
create mode 100644 package-lock.json
create mode 100644 package.json
create mode 100644 server.js

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>git remote add origin https://github.com/Aliza-15/awt-lab.git

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>git branch -M main

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 695, done.
Counting objects: 100% (695/695), done.
Delta compression using up to 8 threads
Compressing objects: 100% (643/643), done.
Writing objects: 100% (695/695), 684.83 KiB | 5.35 MiB/s, done.
Total 695 (delta 120), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (120/120), done.
To https://github.com/Aliza-15/awt-lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\rafia\OneDrive\Documents\NetBeansProjects\AWT LAB\public_html>
```

Aliza-15/awt-lab

https://github.com/Aliza-15/awt-lab

Aliza-15 / awt-lab

Type 7 to search

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

awt-lab Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file <> Code

Aliza-15 First Commit ✓ 0969ace · 18 minutes ago 1 Commit

10 prg	First Commit	18 minutes ago
node_modules	First Commit	18 minutes ago
exp1_static.html	First Commit	18 minutes ago
exp2_responsive.html	First Commit	18 minutes ago
exp3_validation.html	First Commit	18 minutes ago
exp4_dom.html	First Commit	18 minutes ago
exp5_es6.html	First Commit	18 minutes ago
exp6_fetch.html	First Commit	18 minutes ago
exp7_storage.html	First Commit	18 minutes ago

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Deployments 1

github-pages 5 minutes ago

Packages

Pages

https://github.com/Aliza-15/awt-lab/settings/pages

Aliza-15 / **awt-lab**

Type to search

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://aliza-15.github.io/awt-lab/>

Last deployed by Aliza-15 2 minutes ago

Visit site

Unpublish site

Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the main branch. [Learn more about configuring the publishing source for your site.](#)

main / (root) Save

Learn how to [add a Jekyll theme](#) to your site.

Your site was last deployed to the [github-pages](#) environment by the [pages build and deployment](#) workflow. [Learn more about deploying to GitHub Pages using custom workflows](#)